Système de Gestion FAQ ChatBot

Architecture et Implémentation Technique

Documentation Technique Complète

Pipeline d'importation des données depuis Google Sheets
vers une base de données SQLite locale

Auteur : NKOUMOU GERMAIN
Date : 4 janvier 2026
Version : 1.0.0

Table des matières

1 Introduction **2**
1.1 Contexte et Objectifs . . . 2
1.2 Architecture Globale . . . 2

2 Configuration Initiale **2**
2.1 Structure du Projet . . . 2
2.2 Dépendances Système . . . 2

3 Scripts d'Automatisation **3**
3.1 sync.py - Collecte des Données . . . 3
3.2 import_validated.py - Insertion en Base . . . 4

4 Workflow de Gestion des Données **5**
4.1 Cycle de Vie des dnnées . . . 5
4.2 Étapes Détaillées . . . 5

5 Configuration API Google **5**
5.1 Création du Projet Google Cloud . . . 5
5.2 Compte de Service . . . 6
5.3 Partage de la Feuille Google . . . 6

6 Base de Données SQLite **6**
6.1 Structure de la Table FAQ . . . 6
6.2 Schéma de Données . . . 7
6.3 Gestion de la Base . . . 7

7 Journalisation et Monitoring **7**
7.1 Fichier de Logs . . . 7
7.2 Format des Logs . . . 7

8 Déploiement et Maintenance **7**
8.1 Nettoyage des Archives . . . 7
8.2 Sauvegarde . . . 7

9 Dépannage et FAQ Technique **8**
9.1 Problèmes Courants . . . 8
9.2 Vérifications Système . . . 8

1 Introduction

Ce document présente l'architecture complète du **Système de Gestion FAQ ChatBot**. L'objectif principal est d'automatiser la collecte, la validation et l'insertion des questions/réponses depuis un formulaire Google Sheets vers une base de données SQLite locale, afin d'alimenter un chatbot conversationnel.

1.1 Contexte et Objectifs

— **Automatiser** le processus de collecte des données depuis Google Sheets

— **Standardiser** le workflow de validation des données par l'équipe

— **Sécuriser** le stockage des données FAQ dans une base locale

— **Tracer** l'ensemble des opérations via des logs et archives

— **Documenter** l'architecture pour faciliter la maintenance

1.2 Architecture Globale

Le système suit une architecture en pipeline avec les étapes suivantes :

```
[Google Sheets]  [Collecte]  [Validation]  [Base de Données]

   Formulaire     sync.py    Traitement     import_validated.py
     Source      (export)     Manuel          (insertion BD)
```

FIGURE 1 – Flux de données simplifié

2 Configuration Initiale

2.1 Structure du Projet

```
ChatBot/
 config/                # Configuration et clés d'API
    faq-service-key.json
 scripts/               # Scripts Python
    sync.py
    import_validated.py
 data/                  # Données par statut
    pending/
    processing/
    validated/
    archived/
 db/                    # Base de données
    faq.db
 logs/                  # Journaux d'activité
    sync.log
 docs/                  # Documentation
 tests/                 # Tests unitaires
```

2.2 Dépendances Système

```
# Installation des packages Python requis
pip install gspread oauth2client pandas
```

— **Python 3.7+** requis

— **Connexion internet** nécessaire pour la synchronisation

— **Permissions en écriture** sur le système de fichiers

3 Scripts d'Automatisation

3.1 sync.py - Collecte des Données

Ce script est responsable de la collecte automatique des données depuis Google Sheets. Il s'exécute périodiquement et suit le processus suivant :

1. Vérification de la connectivité internet
2. Authentification avec l'API Google Sheets
3. Récupération des nouvelles données
4. Export au format CSV horodaté
5. Journalisation des opérations

Listing 1 – sync.py - Fonction principale

```python
def main():
    # Vérification de la connexion
    if not check_connection():
        log_status("FAILURE - No internet connection")
        return

    try:
        # Configuration des scopes API
        scope = [
            "https://spreadsheets.google.com/feeds",
            "https://www.googleapis.com/auth/drive"
        ]

        # Chargement des credentials
        key_file = Path(__file__).parent.parent / "config" / "faq-service-key.json"
        creds = ServiceAccountCredentials.from_json_keyfile_name(str(key_file), scope)
        client = gspread.authorize(creds)

        # Accès à la feuille
        sheet = client.open("FAQ ChatBot (réponses)").sheet1
        rows = sheet.get_all_records()

        # Création du fichier CSV horodaté
        timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M")
        filename = f"new_data_{timestamp}.csv"
        filepath = Path("data/pending") / filename

        # Export des données
        with filepath.open("w", newline="", encoding="utf-8") as csvfile:
            if rows:
                fieldnames = list(rows[0].keys())
                writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
                writer.writeheader()
```

```
                writer.writerows(rows)

        log_status(f"SUCCESS - Fichier créé: {filename}")

    except Exception as e:
        log_status(f"FAILURE - {str(e)}")
        raise
```

3.2 import_validated.py - Insertion en Base

Ce script traite les fichiers validés par l'équipe et les insère dans la base de données SQLite.

Listing 2 – import_validated.py - Structure

```
def import_validated():
    # Connexion à la base de données
    db_path = Path("db/faq.db")
    conn = sqlite3.connect(str(db_path))
    cursor = conn.cursor()

    # Création de la table si elle n'existe pas
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS faq (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        question TEXT,
        answer TEXT
    )
    """)

    # Traitement des fichiers validés
    validated_dir = Path("data/validated")
    for file in validated_dir.glob("*.csv"):
        df = pd.read_csv(file)

        # Insertion des données
        for _, row in df.iterrows():
            cursor.execute(
                "INSERT INTO faq (question, answer) VALUES (?, ?)",
                (row.get('Question'), row.get('Réponse'))
            )

        conn.commit()

        # Archivage du fichier
        archive_file(file)

    conn.close()
```

4 Workflow de Gestion des Données

4.1 Cycle de Vie des dnnées

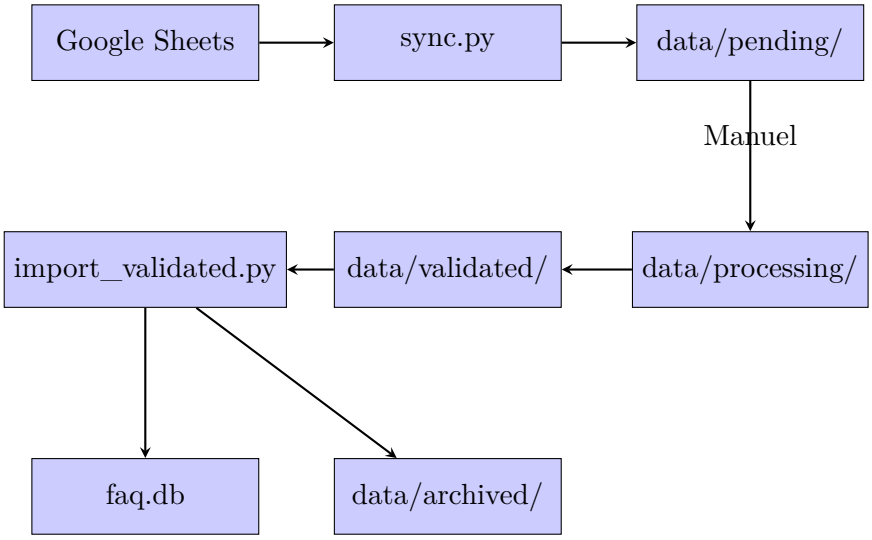

FIGURE 2 – Flux complet de traitement des données

4.2 Étapes Détaillées

1. Collecte Automatique

— Exécution périodique de `sync.py`

— Création de fichiers au format : `new_data_YYYY-MM-DD_HH-MM.csv`

— Stockage initial dans `data/pending/`

2. Validation Manuelle

— Déplacement des fichiers vers `data/processing/`

— Vérification et correction du contenu

— Validation finale et déplacement vers `data/validated/`

3. Insertion en Base

— Exécution de `import_validated.py`

— Lecture de tous les fichiers CSV dans `data/validated/`

— Insertion dans la table `faq` de `faq.db`

— Archivage automatique dans `data/archived/`

5 Configuration API Google

5.1 Création du Projet Google Cloud

1. Accéder à Google Cloud Console
2. Créer un nouveau projet : `faq-chatbot-2026`
3. Activer les APIs :
   — Google Sheets API
   — Google Drive API (nécessaire pour gspread)

5.2 Compte de Service

Compte de Service

Le compte de service permet une authentification sécurisée sans intervention utilisateur. Il est essentiel pour les processus automatisés.

Configuration :

1. Navigation : **IAM & Admin→Service Accounts**
2. Créer un compte : `faq-service`
3. Attribuer le rôle : **Editor** ou **Sheets Editor**
4. Générer une clé JSON : `faq-service-key.json`

Sécurité des Clés

— La clé JSON contient des identifiants sensibles
— Ne jamais commiter dans le dépôt Git
— Ajouter au `.gitignore` : `config/faq-service-key.json`
— Stocker dans un endroit sécurisé hors du projet

5.3 Partage de la Feuille Google

1. Ouvrir la feuille : `FAQ ChatBot (réponses)`
2. Cliquer sur **Partager**
3. Ajouter l'email du compte de service :

   `faq-service@faq-chatbot-2026.iam.gserviceaccount.com`

4. Attribuer les permissions : **Reader** ou **Editor**

6 Base de Données SQLite

6.1 Structure de la Table FAQ

```sql
CREATE TABLE faq (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    question TEXT NOT NULL,
    answer TEXT NOT NULL,
);
```

6.2 Schéma de Données

Colonne	Type	Contrainte	Description
id	INTEGER	PRIMARY KEY	Identifiant unique auto-incrémenté
question	TEXT	NOT NULL	Texte de la question posée
answer	TEXT	NOT NULL	Réponse correspondante

Table 1 – Structure de la table FAQ

6.3 Gestion de la Base

— **Localisation** : `db/faq.db`

— **Interface** : Script Python via sqlite3

— **Sauvegarde** : Recommandée régulièrement

— **Versionnage** : Exclusion du fichier `.db` dans Git

7 Journalisation et Monitoring

7.1 Fichier de Logs

Toutes les opérations de synchronisation sont tracées dans `logs/sync.log` :

```
[2026-01-04 14:30:25] SUCCESS - Fichier créé: new_data_2026-01-04_14-30.
    csv
[2026-01-04 14:30:26] SUCCESS - 15 enregistrements exportés
[2026-01-04 20:30:01] FAILURE - No internet connection
[2026-01-05 08:30:15] SUCCESS - Fichier créé: new_data_2026-01-05_08-30.
    csv
```

7.2 Format des Logs

— **Timestamp** : Date et heure précise

— **Statut** : SUCCESS ou FAILURE

— **Message** : Description détaillée

8 Déploiement et Maintenance

8.1 Nettoyage des Archives

```
# Suppression des archives de plus de 30 jours
find data/archived/ -name "*.csv" -mtime +30 -delete
```

8.2 Sauvegarde

— **Base de données** : Copie régulière de `faq.db`

— **Configuration** : Sauvegarde des clés API

— **Logs** : Archivage mensuel

— **Scripts** : Versionnés dans Git

9 Dépannage et FAQ Technique

9.1 Problèmes Courants

Symptôme	Solution
Erreur d'authentification	Vérifier la validité de la clé JSON
Permission denied sur Google Sheet	Vérifier le partage avec le compte de service
Pas de connexion internet	Attendre la restauration ou vérifier la config réseau
Fichier CSV vide	Vérifier les données dans Google Sheets
Erreur SQLite	Vérifier les permissions en écriture sur db/

Table 2 – Guide de dépannage rapide

9.2 Vérifications Système

1. **Python** : Version 3.7+ installée
2. **Dépendances** : Packages requis installés
3. **Permissions** : Accès en écriture sur les dossiers
4. **Réseau** : Connexion internet active
5. **API Google** : Activées et configurées

Conclusion

Cette documentation présente l'architecture complète du système de gestion FAQ pour ChatBot. L'approche modulaire et automatisée permet :

— Une collecte régulière et fiable des données depuis Google Sheets

— Un processus de validation contrôlé et traçable

— Un stockage sécurisé dans une base de données SQLite locale

— Une maintenance facilitée par une documentation complète

Système opérationnel et documenté
Prêt pour le déploiement en production